Лабораторная работа №3

«Средства безопасности операционной системы MacOS»

Выполнила Авдонина Лада, гр. 23202

Дата оформления отчета: 11.10.2026

Задание №1. Управление пользователями и правами

1. Создание двух пользователей: student и auditor

```
a1234@MacBook-Air-8 ~ % sudo sysadminctl -addUser student -fullName "Student" -password 'StudentPass123!' -home /Users/student -shell /bin/zsh
Password:
2026-10-10 23:25:19.133 sysadminctl[34821:1578162] ----------------------------
2026-10-10 23:25:19.133 sysadminctl[34821:1578162] No clear text password or interactive option was specified (adduser, change/reset password will not allow user to use FDE) !
2026-10-10 23:25:19.133 sysadminctl[34821:1578162] ----------------------------
2026-10-10 23:25:19.343 sysadminctl[34821:1578162] Creating user record…
2026-10-10 23:25:20.220 sysadminctl[34821:1578162] Assigning UID: 503 GID: 20
2026-10-10 23:25:21.255 sysadminctl[34821:1578162] Home directory is assigned (not created!) at /Users/student
a1234@MacBook-Air-8 ~ %  sudo sysadminctl -addUser auditor -fullName "Auditor" -password 'AuditorPass123!' -home /Users/auditor -shell /bin/zsh
Password:
2026-10-11 10:34:31.365 sysadminctl[35085:1591205] ----------------------------
2026-10-11 10:34:31.366 sysadminctl[35085:1591205] No clear text password or interactive option was specified (adduser, change/reset password will not allow user to use FDE) !
2026-10-11 10:34:31.366 sysadminctl[35085:1591205] ----------------------------
2026-10-11 10:34:31.576 sysadminctl[35085:1591205] Creating user record…
2026-10-11 10:34:32.355 sysadminctl[35085:1591205] Assigning UID: 504 GID: 20
2026-10-11 10:34:33.384 sysadminctl[35085:1591205] Home directory is assigned (not created!) at /Users/auditor
```

```
a1234@MacBook-Air-8 ~ % sudo createhomedir -c -u student
creating home directories for (MacBook-Air-8.local)
created (/Users/student)
a1234@MacBook-Air-8 ~ % sudo createhomedir -c -u auditor
creating home directories for (MacBook-Air-8.local)
created (/Users/auditor)
```

Проверка:

```
a1234@MacBook-Air-8 ~ % dscl . -list /Users | grep -E 'student|auditor'
auditor
student
a1234@MacBook-Air-8 ~ % id student
uid=503(student) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),701(com.apple.sharepoint.group.1),702(com.apple.sharepoint.group.2),100(_lpoperator)
a1234@MacBook-Air-8 ~ % id auditor
uid=504(auditor) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),502(security),701(com.apple.sharepoint.group.1),702(com.apple.sharepoint.group.2),100(_lpoperator)
```

```
a1234@MacBook-Air-8 ~ % ls -la /Users/ | grep -E 'student|auditor'
drwxr-x---+ 11 auditor  staff   352 Oct 11 10:58 auditor
drwxr-x---+ 11 student  staff   352 Oct 11 10:58 student
```

2. Создание группы security и добавление в нее auditor

```
a1234@MacBook-Air-8 ~ % sudo dseditgroup -o create security
Password:
a1234@MacBook-Air-8 ~ % sudo dseditgroup -o edit -a auditor -t user security
```

Проверка:

```
a1234@MacBook-Air-8 ~ % dscl . -read /Groups/security
dsAttrTypeNative:record_daemon_version: 8970000
AppleMetaNodeLocation: /Local/Default
GeneratedUID: D6CE5A45-DBDB-4609-AD74-762182AC7EB6
GroupMembers: A5AAD6A1-9A7A-44FD-B423-A79B6FCE0636
GroupMembership: auditor
PrimaryGroupID: 502
RecordName: security
RecordType: dsRecTypeStandard:Groups
a1234@MacBook-Air-8 ~ % id auditor
uid=504(auditor) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),502(security),701(com.apple.sharepoint.group.1),702(com.apple.sharepoint.group.2),100(_lpoperator)
```

3. Изучение прав доступа к файлам: создание файла, изменение его владельца и прав

```
[a1234@MacBook-Air-8 ~ % touch /tmp/testfile
[a1234@MacBook-Air-8 ~ % sudo chown student:security /tmp/testfile
[Password:
[a1234@MacBook-Air-8 ~ % sudo chmod 640 /tmp/testfile
[a1234@MacBook-Air-8 ~ % ls -la /tmp/testfile
 -rw-r-----  1 student  security  0 Oct 11 14:34 /tmp/testfile
```

Проверка:

```
[a1234@MacBook-Air-8 ~ % sudo -u student cat /tmp/testfile && echo "student: OK"
 student: OK
[a1234@MacBook-Air-8 ~ % sudo -u auditor cat /tmp/testfile && echo "auditor: OK (group member)"
 auditor: OK (group member)
[a1234@MacBook-Air-8 ~ % sudo -u a1234   cat /tmp/testfile || echo "a1234: DENIED (not owner, not in group)"
 cat: /tmp/testfile: Permission denied
 a1234: DENIED (not owner, not in group)
```

4. Смена пароля пользователю student

Был введен старый пароль «StudentPass123!»

```
[a1234@MacBook-Air-8 ~ % sudo passwd student
 Changing password for student.
[New password:
[Retype new password:

 ################################### WARNING ###################################
 # This tool does not update the login keychain password.                      #
 # To update it, run `security set-keychain-password` as the user in question, #
 # or as root providing a path to such user's login keychain.                  #
 ###############################################################################
```

Проверка:

```
[a1234@MacBook-Air-8 ~ % dscl . -authonly student 'StudentPass123!' && echo "student password OK"
 student password OK
```

5. Изучение команд umask, chattr, lsattr

chattr и lsattr на macOS отсутствуют. Аналог на Mac - chflags и ls -l0

5.1. umask

umask определяет, какие биты прав вычитаются у новых файлов при создании. Работает одинаково в Linux и macOS.

Смотрим текущее значение:

```
[a1234@MacBook-Air-8 ~ % umask
 022
```

Изменяем и проверяем:

```
a1234@MacBook-Air-8 ~ % umask 027
a1234@MacBook-Air-8 ~ % umask
027
a1234@MacBook-Air-8 ~ % touch /tmp/umask_test
a1234@MacBook-Air-8 ~ % ls -l /tmp/umask_test
-rw-r-----  1 a1234  wheel  0 Oct 11 14:51 /tmp/umask_test
```

Владелец всё, группа читает, остальные ничего (666 - 027 = 640)

```
a1234@MacBook-Air-8 ~ % mkdir /tmp/umask_dir
a1234@MacBook-Air-8 ~ % ls -ld /tmp/umask_dir
drwxr-x---  2 a1234  wheel  64 Oct 11 14:54 /tmp/umask_dir
```

(777 – 027 = 750)

Возвращаем значения по умолчанию:

```
a1234@MacBook-Air-8 ~ % umask 022
a1234@MacBook-Air-8 ~ % umask
022
a1234@MacBook-Air-8 ~ % rm -rf /tmp/umask_test /tmp/umask_dir
```

5.2. chflags и ls -lO

Пересоздадим текстовый файл и проверим флаги:

```
a1234@MacBook-Air-8 ~ % touch /tmp/testfile
a1234@MacBook-Air-8 ~ % sudo chown student:security /tmp/testfile
a1234@MacBook-Air-8 ~ % sudo chmod 640 /tmp/testfile
a1234@MacBook-Air-8 ~ % ls -lO /tmp/testfile
-rw-r-----  1 student  security  - 0 Oct 11 15:01 /tmp/testfile
```

Поставим флаг uchg (аналог chattr +i)

```
a1234@MacBook-Air-8 ~ % sudo chflags uchg /tmp/testfile
Password:
a1234@MacBook-Air-8 ~ % ls -lO /tmp/testfile
-rw-r-----  1 student  security  uchg 0 Oct 11 15:01 /tmp/testfile
```

Проверим, что флаг действительно защищает:

```
a1234@MacBook-Air-8 ~ % echo "x" >> /tmp/testfile
zsh: operation not permitted: /tmp/testfile
a1234@MacBook-Air-8 ~ % sudo sh -c 'echo "root override" >> /tmp/testfile'
sh: /tmp/testfile: Operation not permitted
a1234@MacBook-Air-8 ~ % cat /tmp/testfile
cat: /tmp/testfile: Permission denied
a1234@MacBook-Air-8 ~ % sudo chmod 600 /tmp/testfile
chmod: Unable to change file mode on /tmp/testfile: Operation not permitted
a1234@MacBook-Air-8 ~ % ls -l /tmp/testfile
-rw-r-----  1 student  security  0 Oct 11 15:11 /tmp/testfile
a1234@MacBook-Air-8 ~ % sudo chown root:wheel /tmp/testfile
chown: /tmp/testfile: Operation not permitted
a1234@MacBook-Air-8 ~ % ls -l /tmp/testfile
-rw-r-----  1 student  security  0 Oct 11 15:11 /tmp/testfile
```

Снимаем флаг и убеждаемся, что файл снова доступен для изменения:

```
[a1234@MacBook-Air-8 ~ % sudo chflags nouchg /tmp/testfile

[a1234@MacBook-Air-8 ~ % echo "now writable" >> /tmp/testfile
[a1234@MacBook-Air-8 ~ % cat /tmp/testfile
 now writable
```

6. **Изучение истории команд: history, увеличение размера истории через HISTSIZE в ~/.bashrc.**

На macOS по умолчанию шелл zsh, поэтому файл конфига ~/.zshrc (а не ~/.bashrc).

Исходные значения и текущая история:

```
[a1234@MacBook-Air-8 ~ % echo $HISTSIZE
 2000
[a1234@MacBook-Air-8 ~ % echo $SAVEHIST
 1000
[a1234@MacBook-Air-8 ~ % history | wc -l
       16

[a1234@MacBook-Air-8 ~ % history | tail -10
   790  echo "now writable" >> /tmp/testfile
   791  sudo -u student sh -c 'echo "now writable" >> /tmp/testfile'
   792  cat /tmp/testfile
   793  sudo chmod 666 /tmp/testfile
   794  echo "now writable" >> /tmp/testfile
   795  cat /tmp/testfile
   796  sudo rm -f /tmp/testfile
   797  echo $HISTSIZE
   798  echo $SAVEHIST
   799  history | wc -l
```

Увеличение размера истории через ~/.zshrc:

```
[a1234@MacBook-Air-8 ~ % cat >> ~/.zshrc <<'EOF'
 heredoc> export HISTSIZE=10000
 export SAVEHIST=10000
 setopt INC_APPEND_HISTORY
[EOF
[a1234@MacBook-Air-8 ~ % source ~/.zshrc
[a1234@MacBook-Air-8 ~ % echo $HISTSIZE
 10000
[a1234@MacBook-Air-8 ~ % echo $SAVEHIST
 10000
```

Проверка, что история растет и сохраняется на диск:

```
a1234@MacBook-Air-8 ~ % wc -l ~/.zsh_history
     854 /Users/a1234/.zsh_history
a1234@MacBook-Air-8 ~ % echo "test1"
test1
a1234@MacBook-Air-8 ~ % echo "test2"
test2
a1234@MacBook-Air-8 ~ % history | tail -5
  803  echo $HISTSIZE
  804  echo $SAVEHIST
  805  wc -l ~/.zsh_history
  806  echo "test1"
  807  echo "test2"
```

Задание №2. Настройка UFW (брандмауэр)

Поскольку в macOS нет ufw, для выполнения задания мы будем использовать его встроенные аналоги: Application Firewall (для управления правилами приложений) и pf (Packet Filter) (для фильтрации трафика по портам, как это делает ufw).

1. **Проверка статуса и включение Application Firewall**

Проверим текущий статус:

```
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate
Password:
Firewall is disabled. (State = 0)
```

Включим брандмауэр:

```
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --setglobalstate on
Firewall is enabled. (State = 1)
```

2. **Включение «невидимого режима» (аналог «запретить все входящее»)**

Включим Stealth Mode (компьютер перестает отвечать на посторонние запросы, например, на пинги)

```
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --setstealthmode on
Stealth mode enabled
```

Проверка:

```
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getglobalstate
Firewall is enabled. (State = 1)
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --getstealthmode
Stealth mode enabled
```

3. **Разрешение SSH**

```
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --add /usr/sbin/sshd
Application at path ( /usr/sbin/sshd ) added to firewall
a1234@MacBook-Air-8 ~ % sudo /usr/libexec/ApplicationFirewall/socketfilterfw --unblockapp /usr/sbin/sshd
Incoming connection to the application is permitted
```

4. Настройка PF и блокировка порта 80

Для имитации команды ufw deny 80/tcp были созданы временные правила для pf, блокирующие входящие соединения на порт 80.

Создаем временный файл с правилами:

```
a1234@MacBook-Air-8 ~ % cat > /tmp/lab3.pf.conf <<'EOF'
heredoc> block drop in quick proto tcp from any to any port 80
pass all
EOF
```

Загрузим и включим pf с нашими правилами:

```
a1234@MacBook-Air-8 ~ % sudo pfctl -f /tmp/lab3.pf.conf
pfctl: Use of -f option, could result in flushing of rules
present in the main ruleset added by the system at startup.
See /etc/pf.conf for further details.

No ALTQ support in kernel
ALTQ related functions disabled
a1234@MacBook-Air-8 ~ % sudo pfctl -e
No ALTQ support in kernel
ALTQ related functions disabled
pfctl: pf already enabled
```

Проверим, что правило загружено:

```
a1234@MacBook-Air-8 ~ % sudo pfctl -s rules
No ALTQ support in kernel
ALTQ related functions disabled
block drop in quick proto tcp from any to any port = 80
pass all flags S/SA keep state
```

5. Проверка, что порт 80 действительно заблокирован

В первом окне терминала запускаем "сервер" на порту 80:

```
pass all flags S/SA keep state
a1234@MacBook-Air-8 ~ % sudo nc -l 80
```

Во втором окне терминала попробуем подключиться к нему:

```
a1234@MacBook-Air-8 ~ % nc -vz 127.0.0.1 80
nc: connectx to 127.0.0.1 port 80 (tcp) failed: Operation timed out
```

Значит, наше правило pf блокирует соединение.

Отключим pf и загрузим стандартные правила:

```
a1234@MacBook-Air-8 ~ % sudo pfctl -d
No ALTQ support in kernel
ALTQ related functions disabled
pf disabled
a1234@MacBook-Air-8 ~ % sudo pfctl -f /etc/pf.conf
pfctl: Use of -f option, could result in flushing of rules
present in the main ruleset added by the system at startup.
See /etc/pf.conf for further details.

No ALTQ support in kernel
ALTQ related functions disabled
```

Проверка:

```
a1234@MacBook-Air-8 ~ % sudo nc -l 80

a1234@MacBook-Air-8 ~ % nc -vz 127.0.0.1 80
Connection to 127.0.0.1 port 80 [tcp/http] succeeded!
```

Задание №3. Аудит системы с auditd

В macOS используется не Linux-утилита auditd с auditctl, а две различные подсистемы: OpenBSM (устаревшая, но встроенная) и Unified Logging (современная, рекомендуемая Apple). Для выполнения задания были изучены обе.

1. **Включение и настройка OpenBSM auditd**

Проверка статуса OpenBSM:

```
a1234@MacBook-Air-8 ~ % sudo audit -n
Password:
Error sending trigger:  (ipc/send) invalid destination port
```

Попытка проверить статус аудита выявила ошибку invalid destination port, что указывает на то, что демон auditd не запущен.

Включим auditd через launchctl (аналог systemctl enable):

```
a1234@MacBook-Air-8 ~ % sudo launchctl enable system/com.apple.auditd
a1234@MacBook-Air-8 ~ % sudo launchctl bootstrap system /System/Library/LaunchDaemons/com.apple.auditd.plist
```

Проверим, что аудит запустился:

```
a1234@MacBook-Air-8 ~ % sudo audit -n
Trigger sent.
a1234@MacBook-Air-8 ~ % launchctl list | grep auditd
a1234@MacBook-Air-8 ~ % sudo launchctl list | grep auditd
36560   0       com.apple.auditd
```

2. **Настройка правил аудита (аналог auditctl -w)**

На macOS нет команды auditctl -w для слежения за конкретным файлом. Вместо этого используется конфигурационный файл /etc/security/audit_control, где задаются классы событий (flags)

Посмотрим текущий конфиг:

```
a1234@MacBook-Air-8 ~ % sudo cat /etc/security/audit_control
Password:
cat: /etc/security/audit_control: No such file or directory
```

Файла audit_control нет, создадим сами:

```
a1234@MacBook-Air-8 ~ % sudo ls -la /etc/security/ | grep audit
-r--r--r--   1 root  wheel    652 Oct 20  2023 audit_class
-r--------   1 root  wheel    713 Oct 20  2023 audit_control.example
-r--r--r--   1 root  wheel  26649 Oct 20  2023 audit_event
-r--------   1 root  wheel     77 Oct 20  2023 audit_user
-r-xr-xr-x   1 root  wheel   1326 Oct 20  2023 audit_warn
```

Есть образец /etc/security/audit_control.example, скопируем его

```
a1234@MacBook-Air-8 ~ % sudo ls -la /etc/security/audit_control.example 2>/dev/null && \
sudo cp /etc/security/audit_control.example /etc/security/audit_control && \
sudo chmod 644 /etc/security/audit_control && \
echo "OK — файл создан"
-r--------  1 root  wheel  713 Oct 20  2023 /etc/security/audit_control.example
OK — файл создан
```

Посмотрим содержимое:

```
a1234@MacBook-Air-8 ~ % sudo cat /etc/security/audit_control
#
# DEPRECATION NOTICE
#
# The audit(4) subsystem has been deprecated since macOS 11.0, disabled since
# macOS 14.0, and WILL BE REMOVED in a future version of macOS.  Applications
# that require a security event stream should use the EndpointSecurity(7) API
# instead.
#
# On this version of macOS, you can re-enable audit(4) by renaming or copying
# /etc/security/audit_control.example to /etc/security/audit_control,
# re-enabling the system/com.apple.auditd service by running `launchctl enable
# system/com.apple.auditd` as root, and rebooting.
#
# $P4: //depot/projects/trustedbsd/openbsm/etc/audit_control#8 $
#
dir:/var/audit
flags:lo,aa
minfree:5
naflags:lo,aa
policy:cnt,argv
filesz:2M
expire-after:10M
```

Ключевая строка - flags:lo,aa. В ней нет ad (administrative) и нет ex (exec). Именно их нам и нужно добавить, чтобы аудит отслеживал

создание пользователя и вызов sudo (аналог правил -w /etc/passwd -p wa и -w /usr/bin/sudo -p x из лабы)

Делаем бэкап и правим строку flags:

```
a1234@MacBook-Air-8 ~ % sudo cp /etc/security/audit_control /etc/security/audit_control.bak
a1234@MacBook-Air-8 ~ % sudo sed -i '' '/^flags:/ s/$/,ad,ex/' /etc/security/audit_control
a1234@MacBook-Air-8 ~ % sudo cat /etc/security/audit_control
#
# DEPRECATION NOTICE
#
# The audit(4) subsystem has been deprecated since macOS 11.0, disabled since
# macOS 14.0, and WILL BE REMOVED in a future version of macOS.  Applications
# that require a security event stream should use the EndpointSecurity(7) API
# instead.
#
# On this version of macOS, you can re-enable audit(4) by renaming or copying
# /etc/security/audit_control.example to /etc/security/audit_control,
# re-enabling the system/com.apple.auditd service by running `launchctl enable
# system/com.apple.auditd` as root, and rebooting.
#
# $P4: //depot/projects/trustedbsd/openbsm/etc/audit_control#8 $
#
dir:/var/audit
flags:lo,aa,ad,ex
minfree:5
naflags:lo,aa
policy:cnt,argv
filesz:2M
expire-after:10M
```

Применим конфиг:

```
a1234@MacBook-Air-8 ~ % sudo audit -s
Trigger sent.
a1234@MacBook-Air-8 ~ % sudo audit -n
Trigger sent.
```

3. Выполнить действие - создать testuser

```
a1234@MacBook-Air-8 ~ % sudo createhomedir -c -u testuser
creating home directories for (MacBook-Air-8.local)
created (/Users/testuser)
a1234@MacBook-Air-8 ~ % id testuser
uid=505(testuser) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),701(com.apple.sharepoint.group.1),702(com.apple.sharepoint.group.2),100(_lpoperator)
```

4. Посмотреть события аудита

Проверим, что появились файлы аудита:

```
a1234@MacBook-Air-8 ~ % sudo ls -la /var/audit/
total 18368
drwx------  34 root  wheel     1088 Nov  4  2023 .
drwxr-xr-x  36 root  wheel     1152 Feb 19  2025 ..
-r--r-----   1 root  wheel   253614 Feb  7  2023 20230207051632.crash_recovery
-r--r-----   1 root  wheel     7056 Feb  7  2023 20230207052407.crash_recovery
-r--r-----   1 root  wheel   285544 Apr  7  2023 20230403105401.crash_recovery
-r--r-----   1 root  wheel    34145 Apr  7  2023 20230407062306.crash_recovery
-r--r-----   1 root  wheel   157171 Apr  8  2023 20230407223911.crash_recovery
-r--r-----   1 root  wheel     8923 Apr  8  2023 20230408073006.crash_recovery
-r--r-----   1 root  wheel   371925 Apr 10  2023 20230408160411.crash_recovery
-r--r-----   1 root  wheel    99846 Apr 12  2023 20230410221229.crash_recovery
-r--r-----   1 root  wheel    93099 Apr 13  2023 20230412222234.crash_recovery
-r--r-----   1 root  wheel    33573 Apr 13  2023 20230413070252.crash_recovery
-r--r-----   1 root  wheel    40423 Apr 14  2023 20230414105021.crash_recovery
-r--r-----   1 root  wheel    30217 Apr 16  2023 20230415074007.crash_recovery
-r--r-----   1 root  wheel    28216 Apr 16  2023 20230416014130.crash_recovery
-r--r-----   1 root  wheel    34092 Apr 17  2023 20230416141022.crash_recovery
-r--r-----   1 root  wheel   208797 Apr 23  2023 20230417075733.crash_recovery
-r--r-----   1 root  wheel    58228 Apr 24  2023 20230424010600.crash_recovery
-r--r-----   1 root  wheel    75509 Apr 26  2023 20230425100146.crash_recovery
-r--r-----   1 root  wheel    77020 Apr 29  2023 20230426132241.crash_recovery
-r--r-----   1 root  wheel   128781 May  3  2023 20230429104917.crash_recovery
-r--r-----   1 root  wheel  1323502 Jun 14  2023 20230503083817.crash_recovery
-r--r-----   1 root  wheel    18357 Jun 14  2023 20230614153803.crash_recovery
-r--r-----   1 root  wheel    97331 Jun 16  2023 20230614164508.crash_recovery
-r--r-----   1 root  wheel     8173 Jun 16  2023 20230616123408.crash_recovery
-r--r-----   1 root  wheel     8173 Jun 16  2023 20230616123435.crash_recovery
-r--r-----   1 root  wheel    29229 Jun 17  2023 20230616132006.crash_recovery
-r--r-----   1 root  wheel  1017148 Jul 27  2023 20230616174333.crash_recovery
-r--r-----   1 root  wheel   812128 Aug 27  2023 20230727020859.crash_recovery
-r--r-----   1 root  wheel   199529 Sep  3  2023 20230827143427.crash_recovery
-r--r-----   1 root  wheel   105691 Sep  3  2023 20230903061127.crash_recovery
-r--r-----   1 root  wheel  2076335 Oct 19  2023 20230904014926.crash_recovery
-r--r-----   1 root  wheel  1622070 Nov  4  2023 20231019041102.not_terminated
lrwxr-xr-x   1 root  wheel       40 Oct 19  2023 current -> /var/audit/20231019041102.not_terminated
```

Все файлы в /var/audit/ датированы 2023 годом. Последний файл 20231019041102.not_terminated не завершен (не переключен) с октября 2023. Несмотря на то, что демон auditd отвечает на сигналы (Trigger sent.), новый файл аудита не создается, а попытка его прочитать через auditreduce -c ad /var/audit/current | praudit зависает - утилита ожидает, что файл, помеченный как not_terminated, будет продолжен демоном, чего не происходит.

Это согласуется с DEPRECATION NOTICE: для полноценного запуска auditd требуется перезагрузка системы (...and rebooting). Кроме того, Apple объявила подсистему audit(4) устаревшей с macOS 11.0, отключила её с macOS 14.0 и планирует удалить в будущих версиях, рекомендуя вместо нее EndpointSecurity(7).

На основании DEPRECATION NOTICE и того, что классический auditd в текущей версии macOS не пишет новых trail-файлов без перезагрузки, для выполнения задач аудита лабораторной работы был выбран рекомендованный Apple механизм Unified Logging - утилита log с подкомандами stream (в реальном времени) и show (по истории). Он функционально покрывает задачи, для которых в Linux используются auditctl -w и ausearch -k.

В первом окне терминала был запущен мониторинг событий, связанных с процессами управления пользователями:

```
a1234@MacBook-Air-8 ~ % log stream --predicate 'process == "sysadminctl" OR process == "createhomedir"' --style compact
Filtering the log data using "process == "sysadminctl" OR process == "createhomedir""
```

Во втором окне была выполнена последовательность операций - удаление, повторное создание и создание домашнего каталога testuser:

```
a1234@MacBook-Air-8 ~ % sudo sysadminctl -deleteUser testuser
Password:
2026-10-11 17:24:25.816 sysadminctl[36968:1720834] ----------------------------
2026-10-11 17:24:25.816 sysadminctl[36968:1720834] No clear text password or interactive option was specified (adduser, change/reset password will not allow user to use FDE) !
2026-10-11 17:24:25.816 sysadminctl[36968:1720834] ----------------------------
2026-10-11 17:24:25.896 sysadminctl[36968:1720834] Killing all processes for UID 505
2026-10-11 17:24:25.897 sysadminctl[36968:1720834] Removing testuser's home at /Users/testuser
2026-10-11 17:24:25.930 sysadminctl[36968:1720834] Deleting Public share point for testuser
2026-10-11 17:24:25.941 sysadminctl[36968:1720834] Deleting record for testuser
2026-10-11 17:24:25.960 sysadminctl[36968:1720834] AOSKit INFO: Disabling BTMM for user, no zone found for uid=505, usersToZones: (null)
a1234@MacBook-Air-8 ~ % sudo sysadminctl -addUser testuser -password 'TestPass123!' -home /Users/testuser
2026-10-11 17:24:43.373 sysadminctl[36982:1721153] ----------------------------
2026-10-11 17:24:43.373 sysadminctl[36982:1721153] No clear text password or interactive option was specified (adduser, change/reset password will not allow user to use FDE) !
2026-10-11 17:24:43.373 sysadminctl[36982:1721153] ----------------------------
2026-10-11 17:24:43.571 sysadminctl[36982:1721153] Creating user record…
2026-10-11 17:24:44.470 sysadminctl[36982:1721153] Assigning UID: 505 GID: 20
2026-10-11 17:24:45.504 sysadminctl[36982:1721153] Home directory is assigned (not created!) at /Users/testuser
a1234@MacBook-Air-8 ~ % sudo createhomedir -c -u testuser
creating home directories for (MacBook-Air-8.local)
created (/Users/testuser)
```

В первом окне в реальном времени были зафиксированы все ключевые события:

```
2026-10-11 17:24:25.896 Df sysadminctl[36968:1a4202] Killing all processes for UID 505
2026-10-11 17:24:25.897 Df sysadminctl[36968:1a4202] Removing testuser's home at /Users/testuser
2026-10-11 17:24:25.930 Df sysadminctl[36968:1a4202] Deleting Public share point for testuser

2026-10-11 17:24:25.941 Df sysadminctl[36968:1a4202] Deleting record for testuser

2026-10-11 17:24:43.571 Df sysadminctl[36982:1a4341] Creating user record…

2026-10-11 17:24:44.470 Df sysadminctl[36982:1a4341] Assigning UID: 505 GID: 20

2026-10-11 17:24:45.504 Df sysadminctl[36982:1a4341] Home directory is assigned (not created!) at /Users/testuser
```

В отличие от log stream, который показывает события в реальном времени, команда log show извлекает события из архива журнала. Для отсечения служебного шума (проверки групп, XPC-соединения, запросы к Directory Services) вывод был отфильтрован по ключевым событиям управления учетными записями:

```
a1234@MacBook-Air-8 ~ % log show --predicate 'process == "sysadminctl"' --last 30m --style compact | grep -E 'Creating user record|Assigning UID|Home directory|Deleting record|Removing .* home|Deleting Public|Create a new record|Change password for record'
2026-10-11 17:24:25.897 Df sysadminctl[36968:1a4202] Removing testuser's home at /Users/testuser
2026-10-11 17:24:25.930 Df sysadminctl[36968:1a4202] Deleting Public share point for testuser
2026-10-11 17:24:25.941 Df sysadminctl[36968:1a4202] Deleting record for testuser
2026-10-11 17:24:43.571 Df sysadminctl[36982:1a4341] Creating user record…
2026-10-11 17:24:43.604 A  sysadminctl[36982:1a4341] (CFOpenDirectory) Create a new record in a node
2026-10-11 17:24:43.622 A  sysadminctl[36982:1a4341] (CFOpenDirectory) Change password for record
2026-10-11 17:24:44.470 Df sysadminctl[36982:1a4341] Assigning UID: 505 GID: 20
2026-10-11 17:24:44.515 A  sysadminctl[36982:1a4341] (CFOpenDirectory) Change password for record
2026-10-11 17:24:45.504 Df sysadminctl[36982:1a4341] Home directory is assigned (not created!) at /Users/testuser
```

В выводе четко прослеживается полный жизненный цикл учетной записи: удаление предыдущей (Removing testuser's home, Deleting record for testuser) и создание новой (Creating user record..., Create a new record in a node, Change password for record, Assigning UID: 505 GID: 20, Home directory is assigned ... /Users/testuser).

Временные метки в логе полностью соответствуют моментам выполнения команд, что подтверждает синхронность и корректность фиксации событий. Дополнительно зафиксированы события уровня Directory Services (Create a new record in a node, Change password for record), которые в Linux-варианте соответствовали бы аудиту операций над /etc/passwd.

5. **Отслеживание выполнения sudo**

В Linux для отслеживания каждого вызова sudo используется правило sudo auditctl -w /usr/bin/sudo -p x -k sudo_usage, где -w - путь к наблюдаемому файлу, -p x - режим «execute», -k sudo_usage - ключ для последующего поиска через ausearch.

В macOS аналогичная задача решается через Unified Logging - фильтр по имени процесса sudo. Это эквивалент «watch»-правила: любое выполнение sudo автоматически фиксируется в журнале подсистемы.

Во втором окне запустим мониторинг sudo в реальном времени:

```
a1234@MacBook-Air-8 ~ % log stream --predicate 'process == "sudo"' --style compact
Filtering the log data using "process == "sudo""
```

В первом окне выполняем:

```
a1234@MacBook-Air-8 ~ % sudo whoami
Password:
root
a1234@MacBook-Air-8 ~ % sudo id
uid=0(root) gid=0(wheel) groups=0(wheel),1(daemon),2(kmem),3(sys),4(tty),5(operator),8(procview),9(procmod),12(everyone),20(staff),29(certusers),61(localaccounts),80(admin),701(com.apple.sharepoint.group.1),702(com.apple.sharepoint.group.2),33(_appstore),98(_lpadmin),100(_lpoperator),204(_developer),250(_analyticsusers),395(com.apple.access_ftp),398(com.apple.access_screensharing),399(com.apple.access_ssh),400(com.apple.access_remote_ae),101(access_bpf)
a1234@MacBook-Air-8 ~ % sudo ls /tmp
com.adobe.AdobeIPCBroker.ctrl-a1234     com.apple.launchd.N9xpsR0834            com.apple.launchd.Vad2pF8P3G            lab3.pf.conf                            tmp-mount-Cba955
a1234@MacBook-Air-8 ~ % sudo date
Sun Oct 11 17:44:07 +07 2026
```

Каждый вызов sudo whoami, sudo id, sudo ls /tmp, sudo date, выполненный во втором окне, породил отдельный процесс sudo, зафиксированный в журнале.

```
Timestamp               Ty Process[PID:TID]
2026-10-11 17:43:39.882 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve User by ID
2026-10-11 17:43:39.882 A  sudo[37178:1a73df] (libsystem_info.dylib) Performance impact - Resolve user group list (>17 groups)
2026-10-11 17:43:39.882 Df sudo[37178:1a73df] (libsystem_info.dylib) Too many groups requested (2147483647).  Can cause performance issues when network directories are involved
2026-10-11 17:43:39.888 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve User by Name
2026-10-11 17:43:39.888 Df sudo[37178:1a73df] Reading config
2026-10-11 17:43:39.889 Df sudo[37178:1a73df] Using original path (null)
2026-10-11 17:43:39.890 A  sudo[37178:1a73df] (libsystem_info.dylib) Performance impact - Resolve user group list (>17 groups)
2026-10-11 17:43:39.890 Df sudo[37178:1a73df] (libsystem_info.dylib) Too many groups requested (2147483647).  Can cause performance issues when network directories are involved
2026-10-11 17:43:39.892 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
```

Историческая выборка (аналог ausearch -k sudo_usage):

```
a1234@MacBook-Air-8 ~ % log show --predicate 'process == "sudo"' --last 10m --style compact \
  | grep -E 'USER|TTY|COMMAND|sudo|pam|authentication|sudoers' | head -30
2026-10-11 17:43:39.881 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve User by ID
2026-10-11 17:43:39.882 A  sudo[37178:1a73df] (libsystem_info.dylib) Performance impact - Resolve user group list (>17 groups)
2026-10-11 17:43:39.882 Df sudo[37178:1a73df] (libsystem_info.dylib) Too many groups requested (2147483647).  Can cause performance issues when network directories are involved
2026-10-11 17:43:39.888 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve User by Name
2026-10-11 17:43:39.888 Df sudo[37178:1a73df] Reading config
2026-10-11 17:43:39.889 Df sudo[37178:1a73df] Using original path (null)
2026-10-11 17:43:39.890 A  sudo[37178:1a73df] (libsystem_info.dylib) Performance impact - Resolve user group list (>17 groups)
2026-10-11 17:43:39.890 Df sudo[37178:1a73df] (libsystem_info.dylib) Too many groups requested (2147483647).  Can cause performance issues when network directories are involved
2026-10-11 17:43:39.892 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.892 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.892 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.892 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.892 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.893 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.893 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.893 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.893 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.893 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.894 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.894 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.894 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.894 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.894 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.894 A  sudo[37178:1a73df] (libsystem_info.dylib) Retrieve Group by ID
2026-10-11 17:43:39.920 A  sudo[37178:1a73df] (CFOpenDirectory) Open a given node
2026-10-11 17:43:39.921 Df sudo[37178:1a73df] [com.apple.xpc:connection] [0x60000289c000] activating connection: mach=true listener=false peer=false name=com.apple.system.opendirectoryd.api
2026-10-11 17:43:39.923 A  sudo[37178:1a73df] (CFOpenDirectory) Copy nodenames that are currently unreachable
2026-10-11 17:43:39.923 A  sudo[37178:1a73df] (CFOpenDirectory) Retrieve record from node
```

Задание №4. AppArmor и мандатный контроль доступа

В macOS для мандатного контроля доступа используется встроенный механизм sandbox-exec (Seatbelt) с профилями на языке SBPL (Sandbox Profile Language). Это аналог AppArmor в Linux. Несмотря на то, что Apple помечает sandbox-exec как deprecated, он полностью функционален в современных версиях macOS.

1. Проверка статуса

Проверка доступности sandbox-exec (аналог aa-status):

```
a1234@MacBook-Air-8 ~ % which sandbox-exec
/usr/bin/sandbox-exec
```

2. Проверка профилей

Просмотр списка системных профилей (аналог ls /etc/apparmor.d/):

```
a1234@MacBook-Air-8 ~ % ls /System/Library/Sandbox/Profiles/ | head -20
ASPCarryLog.sb
NANDTaskScheduler.sb
WeatherKitAuthService.sb
WiFiVelocityAgent.sb
airlock.sb
application.sb
appsandbox-common.sb
apsd.sb
betaenrollmentd.sb
blastdoor.sb
bluetoothuserd.sb
bsd.sb
calaccessd.sb
cloudpaird.sb
cmioextension.sb
colorsyncd.sb
com.apple.ANECompilerService.sb
com.apple.AirPlayXPCHelper.sb
com.apple.AppSSOAgent.login.sb
com.apple.AppSSOAgent.sb
```

```
a1234@MacBook-Air-8 ~ % ls /usr/share/sandbox/ | head -20
airportd.sb
awdd.sb
bluetoothd.sb
com.apple.CloudDocsDaemon.ciconia.sb
com.apple.CommCenter.sb
com.apple.FontValidator.sb
com.apple.USBAgent.sb
com.apple.atsd.internal.sb
com.apple.atsd.support.sb
com.apple.bird.sb
com.apple.bootinstalld.sb
com.apple.ckdiscretionaryd.sb
com.apple.cloudd.sb
com.apple.corespotlightservice.sb
com.apple.fontd.internal.sb
com.apple.fontd.support.sb
com.apple.genatsdb.internal.sb
com.apple.mobileassetd.sb
com.apple.msrpc.lsarpc.sb
com.apple.msrpc.mdssvc.sb
```

3. Создание тестового приложения

Создадим «секретный» файл - его мы будем пытаться прочитать под песочницей:

```
a1234@MacBook-Air-8 ~ % echo "TOP SECRET DATA" > /tmp/secret.txt
a1234@MacBook-Air-8 ~ % chmod 644 /tmp/secret.txt
a1234@MacBook-Air-8 ~ % cat /tmp/secret.txt
TOP SECRET DATA
```

Теперь - сам тестовый скрипт. Он делает две вещи: пытается прочитать /tmp/secret.txt и создать файл /tmp/testapp_output. В обоих случаях печатает результат:

```
a1234@MacBook-Air-8 ~ % cat > /tmp/testapp.sh <<'EOF'
#!/bin/zsh
echo "=== testapp running ==="
if cat /tmp/secret.txt >/dev/null 2>&1; then
    echo "READ  : ALLOWED"
else
    echo "READ  : DENIED"
fi
if touch /tmp/testapp_output 2>/dev/null; then
    echo "WRITE : ALLOWED"
else
    echo "WRITE : DENIED"
fi
EOF
a1234@MacBook-Air-8 ~ % chmod +x /tmp/testapp.sh
a1234@MacBook-Air-8 ~ % ls -la /tmp/testapp.sh /tmp/secret.txt
-rw-r--r--  1 a1234  wheel   16 Oct 11 18:25 /tmp/secret.txt
-rwxr-xr-x  1 a1234  wheel  256 Oct 11 18:26 /tmp/testapp.sh
```

Проверка:

```
a1234@MacBook-Air-8 ~ % /tmp/testapp.sh
=== testapp running ===
READ  : ALLOWED
WRITE : ALLOWED
```

4. Профиль в режиме «жалобы» (complain) - аналог aa-complain

Создан профиль SBPL с директивой (trace "/tmp/testapp_trace.log"), которая по документации должна соответствовать режиму complain в AppArmor (логирование без блокировки):

```
a1234@MacBook-Air-8 ~ % cat > /tmp/testapp_complain.sb <<'EOF'
(version 1)
(trace "/tmp/testapp_trace.log")
(allow default)
(deny file-read* (literal "/tmp/secret.txt") (literal "/private/tmp/secret.txt"))
(deny file-write* (literal "/tmp/testapp_output") (literal "/private/tmp/testapp_output"))
EOF
a1234@MacBook-Air-8 ~ % cat /tmp/testapp_complain.sb
(version 1)
(trace "/tmp/testapp_trace.log")
(allow default)
(deny file-read* (literal "/tmp/secret.txt") (literal "/private/tmp/secret.txt"))
(deny file-write* (literal "/tmp/testapp_output") (literal "/private/tmp/testapp_output"))
```

Запуск:

```
a1234@MacBook-Air-8 ~ % sandbox-exec -f /tmp/testapp_complain.sb /tmp/testapp.sh
=== testapp running ===
READ  : DENIED
WRITE : DENIED
```

Результат DENIED / DENIED показывает, что директива (trace) не переключила профиль в режим логирования - запреты сработали немедленно, как в режиме enforce. Дополнительная проверка подтверждает, что файл трассировки не создан:

```
a1234@MacBook-Air-8 ~ % ls -la /tmp/testapp_trace.log 2>&1
ls: /tmp/testapp_trace.log: No such file or directory
a1234@MacBook-Air-8 ~ % ls -la /tmp/testapp_output 2>/dev/null && echo "ОШИБКА: файл создан" || echo "OK: файл НЕ создан"
OK: файл НЕ создан
```

В macOS отсутствует режим complain как таковой. Директива (trace) присутствует в синтаксисе SBPL, но ядром игнорируется. Любой заданный профиль с правилами (deny ...) немедленно применяется в режиме принуждения. Это принципиальное отличие от Linux, где aa-complain позволяет безопасно отлаживать профиль, наблюдая за срабатываниями без блокировки приложения.

5. Режим «принуждения» (enforce) - аналог aa-enforce

Создан профиль без (trace):

```
a1234@MacBook-Air-8 ~ % cat > /tmp/testapp_enforce.sb <<'EOF'
(version 1)
(allow default)
(deny file-read* (literal "/tmp/secret.txt") (literal "/private/tmp/secret.txt"))
(deny file-write* (literal "/tmp/testapp_output") (literal "/private/tmp/testapp_output"))
EOF
a1234@MacBook-Air-8 ~ % cat /tmp/testapp_enforce.sb
(version 1)
(allow default)
(deny file-read* (literal "/tmp/secret.txt") (literal "/private/tmp/secret.txt"))
(deny file-write* (literal "/tmp/testapp_output") (literal "/private/tmp/testapp_output"))
```

Запуск:

```
a1234@MacBook-Air-8 ~ % sandbox-exec -f /tmp/testapp_enforce.sb /tmp/testapp.sh
=== testapp running ===
READ  : DENIED
WRITE : DENIED

a1234@MacBook-Air-8 ~ % ls -la /tmp/testapp_output 2>/dev/null && echo "ОШИБКА: файл создан" || echo "OK: файл НЕ создан"
OK: файл НЕ создан
```

Запрещенные в профиле файловые операции блокируются, а остальные (запись в stdout) работают, приложение не падает.

6. **Логи песочницы (аналог dmesg | grep apparmor)**

В Linux логи AppArmor доступны через dmesg | grep apparmor. В macOS они записываются ядром в Unified Logging под подсистемой Sandbox. Во втором окне терминала был запущен поток наблюдения:

```
a1234@MacBook-Air-8 ~ % log stream --predicate 'sender=="Sandbox" OR eventMessage CONTAINS "deny"' --style compact
Filtering the log data using "sender == "Sandbox" OR composedMessage CONTAINS "deny""
```

В первом окне под enforce-профилем снова был запущен testapp.sh. В логе появились прямые записи об отказах на уровне ядра:

```
a1234@MacBook-Air-8 ~ % log show --predicate 'sender=="Sandbox"' --last 5m --style compact \
  | grep -E 'secret\.txt|testapp_output'
2026-10-11 18:45:17.674 E  kernel[0:1b0aba] (Sandbox) Sandbox: cat(37878) deny(1) file-read-data /private/tmp/secret.txt
2026-10-11 18:45:17.678 E  kernel[0:1b0abc] (Sandbox) Sandbox: touch(37879) deny(1) file-write-create /private/tmp/testapp_output
```

Задание №5. Автоматические обновления безопасности

В Ubuntu за автоматические обновления безопасности отвечает пакет unattended-upgrades, настраиваемый через /etc/apt/apt.conf.d/50unattended-upgrades. В macOS аналогичную функцию выполняет встроенная служба Software Update (демон softwareupdated), конфигурация которой хранится в plist-файле /Library/Preferences/com.apple.SoftwareUpdate.plist.

1. **Проверка статуса «unattended-upgrades» (аналог systemctl status)**

В macOS демон обновлений – softwareupdated, проверяем:

```
a1234@MacBook-Air-8 ~ % launchctl list | grep -i softwareupdate
-       -9      com.apple.SoftwareUpdateNotificationManager
a1234@MacBook-Air-8 ~ % ps aux | grep -i softwareupdated | grep -v grep
_softwareupdate  29825   0.2  0.3 412703488  23376   ??  Ss   Sat09AM   2:13.59 /System/Library/CoreServices/Software Update.app/Contents/Resources/softwareupdated
a1234@MacBook-Air-8 ~ % sw_vers
ProductName:            macOS
ProductVersion:         14.1
BuildVersion:           23B74
```

2. **Проверка конфигурации автообновлений (аналог cat 50unattended-upgrades)**

В Ubuntu конфиг - это текстовый файл. В macOS - plist с ключами. Читаем его через defaults:

```
a1234@MacBook-Air-8 ~ % sudo defaults read /Library/Preferences/com.apple.SoftwareUpdate
Password:
{
    DidSkipBackgroundDownloadLackingDiskSpace = 0;
    LastAttemptBuildVersion = "14.1 (23B74)";
    LastAttemptSystemVersion = "14.1 (23B74)";
    LastBackgroundSuccessfulDate = "2026-10-09 13:54:43 +0000";
    LastFullSuccessfulDate = "2026-10-10 02:55:10 +0000";
    LastRecommendedMajorOSBundleIdentifier = "";
    LastRecommendedUpdatesAvailable = 1;
    LastResultCode = 2;
    LastSessionSuccessful = 1;
    LastSuccessfulDate = "2026-10-10 03:00:05 +0000";
    LastUpdatesAvailable = 1;
    PrimaryLanguages =     (
        "ru-RU",
        ru,
        "ru-US"
    );
    RecommendedUpdates =     (
                {
            "Display Name" = "macOS Sonoma 14.8.9";
            "Display Version" = "14.8.9";
            Identifier = "MSU_UPDATE_23J631_patch_14.8.9_minor";
            MobileSoftwareUpdate = 1;
            "Product Key" = "MSU_UPDATE_23J631_patch_14.8.9_minor";
        }
```

Ключевые поля:

LastFullSuccessfulDate - дата последней успешной полной проверки обновлений

LastSuccessfulDate - дата последней успешной синхронизации с сервером Apple

LastSessionSuccessful - последняя сессия проверки завершилась без ошибок

LastRecommendedUpdatesAvailable - система обнаружила одно рекомендуемое обновление

RecommendedUpdates - список доступных обновлений, найденных автоматически

Эти данные показывают, что подсистема Software Update работает в фоне: система автоматически проверяет наличие обновлений (последняя проверка 10.10.2026) и уже обнаружила апдейт безопасности macOS Sonoma 14.8.9, готовый к установке. Это функциональный аналог работы unattended-upgrades в Ubuntu, где аналогичная информация фиксируется в /var/log/unattended-upgrades/unattended-upgrades.log.

3. Автоматические обновления безопасности

Явно включаем автообновления безопасности:

```
a1234@MacBook-Air-8 ~ % sudo defaults write /Library/Preferences/com.apple.SoftwareUpdate AutomaticCheckEnabled -bool true
Password:
a1234@MacBook-Air-8 ~ % sudo defaults write /Library/Preferences/com.apple.SoftwareUpdate AutomaticDownload -bool true
a1234@MacBook-Air-8 ~ % sudo defaults write /Library/Preferences/com.apple.SoftwareUpdate CriticalUpdateInstall -bool true
a1234@MacBook-Air-8 ~ % sudo defaults write /Library/Preferences/com.apple.SoftwareUpdate ConfigDataInstall -bool true
a1234@MacBook-Air-8 ~ % sudo defaults write /Library/Preferences/com.apple.SoftwareUpdate AutomaticallyInstallMacOSUpdates -bool true
a1234@MacBook-Air-8 ~ % sudo defaults write /Library/Preferences/com.apple.SoftwareUpdate.plist SplatEnabled -bool true
```

Проверяем:

```
a1234@MacBook-Air-8 ~ % for k in AutomaticCheckEnabled AutomaticDownload CriticalUpdateInstall ConfigDataInstall AutomaticallyInstallMacOSUpdates; do
  printf "%-35s = %s\n" "$k" "$(sudo defaults read /Library/Preferences/com.apple.SoftwareUpdate "$k" 2>/dev/null)"
done

echo "--- Background Security Improvements ---"
printf "%-35s = %s\n" "SplatEnabled" "$(sudo defaults read /Library/Preferences/com.apple.SoftwareUpdate.plist SplatEnabled 2>/dev/null)"
AutomaticCheckEnabled               = 1
AutomaticDownload                   = 1
CriticalUpdateInstall               = 1
ConfigDataInstall                   = 1
AutomaticallyInstallMacOSUpdates    = 1
--- Background Security Improvements ---
SplatEnabled                        = 1
```

4. Ручное обновление (аналог apt update && apt upgrade)

Проверяем доступные обновления (apt update):

```
a1234@MacBook-Air-8 ~ % sudo softwareupdate --list
Password:
Software Update Tool

Finding available software
No new software available.
```

Система успешно обратилась к серверу Apple и получила ответ, что обновлений для текущей конфигурации нет. Это не означает, что автообновления не работают, наоборот, команда --list подтверждает, что механизм проверки активен и отвечает. Дополнительно в plist зафиксировано, что при предыдущем фоновом сканировании (10.10.2026) система автоматически обнаружила обновление macOS Sonoma 14.8.9 (LastRecommendedUpdatesAvailable = 1,

RecommendedUpdates), то есть подсистема автообновлений macOS действительно выполняет проверки в фоне без участия пользователя.

Аналог apt-истории - softwareupdate –history - покажет все установленные обновления, включая security-компоненты:

```
a1234@MacBook-Air-8 ~ % softwareupdate --history
Display Name                                          Version    Date
------------                                          -------    ----
Command Line Tools for Xcode                          14.3       09/02/2023, 16:48:58
macOS 14.1                                            14.1       11/04/2023, 09:24:24
Command Line Tools for Xcode                          15.3       09/27/2026, 15:55:09
```

Задание №6. Защита удаленного доступа (SSH)

В macOS, как и в Linux, используется сервер OpenSSH. Синтаксис /etc/ssh/sshd_config полностью совпадает. Отличается только управление службой: вместо systemctl применяется launchctl и systemsetup.

6.1 Настройка SSH-сервера

1. Включение SSH-сервера (аналог systemctl enable --now ssh)

```
a1234@MacBook-Air-8 ~ % sudo systemsetup -getremotelogin
Password:
Remote Login: Off
a1234@MacBook-Air-8 ~ % sudo systemsetup -setremotelogin on
a1234@MacBook-Air-8 ~ % sudo systemsetup -getremotelogin
Remote Login: On
```

2. Проверка, что SSHd слушает порт

```
a1234@MacBook-Air-8 ~ % sudo launchctl list | grep ssh
-       0       com.openssh.sshd
a1234@MacBook-Air-8 ~ % sudo lsof -iTCP:22 -sTCP:LISTEN
COMMAND PID USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
launchd   1 root   43u  IPv6 0xc619595bd086956b      0t0  TCP *:ssh (LISTEN)
launchd   1 root   44u  IPv4 0xc61959609d31f98b      0t0  TCP *:ssh (LISTEN)
launchd   1 root   45u  IPv6 0xc619595bd086956b      0t0  TCP *:ssh (LISTEN)
launchd   1 root   46u  IPv4 0xc61959609d31f98b      0t0  TCP *:ssh (LISTEN)
```

Особенность macOS: в отличие от Linux, где процесс sshd постоянно висит в состоянии LISTEN, в macOS используется механизм socket activation. Порт 22 слушает системный менеджер launchd (PID 1), а сам sshd порождается по требованию при поступлении входящего соединения и завершается после его обработки. Это уменьшает поверхность атаки (нет постоянно работающего сетевого демона) и снижает потребление ресурсов.

Сервис com.openssh.sshd зарегистрирован в launchd (статус 0 = последний запуск успешен, активного PID сейчас нет, что и ожидается для socket-activated сервиса).

3. Настройка /etc/ssh/sshd_config

Сделан бэкап:

```
a1234@MacBook-Air-8 ~ % sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
a1234@MacBook-Air-8 ~ % ls -la /etc/ssh/sshd_config*
-rw-r--r--  1 root  wheel  3610 Mar 18  2025 /etc/ssh/sshd_config
-rw-r--r--  1 root  wheel  3610 Oct 11 19:55 /etc/ssh/sshd_config.bak

/etc/ssh/sshd_config.d:
total 8
drwxr-xr-x  3 root  wheel   96 Oct 20  2023 .
drwxr-xr-x  8 root  wheel  256 Oct 11 19:55 ..
-rw-r--r--  1 root  wheel  133 Oct 20  2023 100-macos.conf
```

Смотрим текущие значения:

```
a1234@MacBook-Air-8 ~ % sudo sshd -T | grep -E '^(permitrootlogin|passwordauthentication|pubkeyauthentication|port|maxauthtries|logingracetime|allowusers)'
Error reading managed configuration (2: No such file or directory). Proceeding with default configuration.
port 22
logingracetime 120
maxauthtries 6
permitrootlogin without-password
pubkeyauthentication yes
passwordauthentication yes
```

Значения по умолчанию не соответствуют требованиям безопасности: пароль разрешён, порт стандартный, число попыток 6, время на ввод 120 секунд, root может входить по ключу, ограничений по пользователям нет.

Вносим изменения:

```
a1234@MacBook-Air-8 ~ % sudo tee -a /etc/ssh/sshd_config > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
Port 2222
MaxAuthTries 3
LoginGraceTime 30
AllowUsers student auditor
EOF
a1234@MacBook-Air-8 ~ % sudo tail -15 /etc/ssh/sshd_config
#Match User anoncvs
#       X11Forwarding no
#       AllowTcpForwarding no
#       PermitTTY no
#       ForceCommand cvs server

# XAuthLocation added by XQuartz (https://www.xquartz.org)
XAuthLocation /opt/X11/bin/xauth
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
Port 2222
MaxAuthTries 3
LoginGraceTime 30
AllowUsers student auditor
```

Финальная конфигурация:

```
a1234@MacBook-Air-8 ~ % sudo sshd -T | grep -E '^(permitrootlogin|passwordauthentication|pubkeyauthentication|port|maxauthtries|logingracetime|allowusers)'
Error reading managed configuration (2: No such file or directory). Proceeding with default configuration.
port 2222
logingracetime 30
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
allowusers student
allowusers auditor
```

Мы дошли до пункта 5 (перезапуск SSH), но перезапускать сейчас нельзя, потому что student ещё нет ключа в authorized_keys, и зайти по SSH никто не сможет

6.2 Настройка ключевой аутентификации

1. Генерация пары ключей

```
a1234@MacBook-Air-8 ~ % ls -la ~/.ssh/id_ed25519_student 2>/dev/null
a1234@MacBook-Air-8 ~ % ssh-keygen -t ed25519 -C "student@lab" -f ~/.ssh/id_ed25519_student -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /Users/a1234/.ssh/id_ed25519_student
Your public key has been saved in /Users/a1234/.ssh/id_ed25519_student.pub
The key fingerprint is:
SHA256:IQmTHX5uXPhyjPmQK0f/5O9z8KOsJFKw7+OVxCPcHFQ student@lab
The key's randomart image is:
+--[ED25519 256]--+
|    oo..   ..E   |
|    .+.. ..      |
|      +.+ ..     |
|       =+O+ .    |
|       .S++*     |
|       ooBo o .  |
|      ..ooo+.  o |
|       oo.++. ..o|
|        .o..+=+o.|
+----[SHA256]-----+
```

Проверяем, что ключ есть:

```
a1234@MacBook-Air-8 ~ % ls -la ~/.ssh/id_ed25519_student*
-rw-------  1 a1234  staff  399 Oct 11 20:12 /Users/a1234/.ssh/id_ed25519_student
-rw-r--r--  1 a1234  staff   93 Oct 11 20:12 /Users/a1234/.ssh/id_ed25519_student.pub
```

2. Копирование публичного ключа на сервер

На macOS ssh-copy-id работает нестабильно из-за особенностей прав на системные команды. Сделаем копирование вручную:

```
a1234@MacBook-Air-8 ~ % sudo mkdir -p /Users/student/.ssh
Password:
a1234@MacBook-Air-8 ~ % sudo chown -R student:staff /Users/student/.ssh
a1234@MacBook-Air-8 ~ % sudo chmod 700 /Users/student/.ssh
a1234@MacBook-Air-8 ~ % sudo cp ~/.ssh/id_ed25519_student.pub /Users/student/.ssh/authorized_keys
a1234@MacBook-Air-8 ~ % sudo chown student:staff /Users/student/.ssh/authorized_keys
a1234@MacBook-Air-8 ~ % sudo chmod 600 /Users/student/.ssh/authorized_keys
a1234@MacBook-Air-8 ~ % sudo ls -la /Users/student/.ssh/
total 8
drwx------   3 student  staff   96 Oct 11 20:15 .
drwxr-x---+ 12 student  staff  384 Oct 11 20:14 ..
-rw-------   1 student  staff   93 Oct 11 20:15 authorized_keys
a1234@MacBook-Air-8 ~ % sudo cat /Users/student/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKOAtBMOSp5W+EXCyPShlH+FEEi4w5aHj3iob4kHRxzJ student@lab
```

3. Проверка входа по ключу на СТАРОМ порту 22

Делаем это, чтобы до перезапуска проверить, что ключ работает.

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o PreferredAuthentications=publickey \
    -i ~/.ssh/id_ed25519_student student@127.0.0.1 whoami
Warning: Permanently added '127.0.0.1' (ED25519) to the list of known hosts.
Connection closed by 127.0.0.1 port 22
```

Первая попытка входа по ключу завершилась ошибкой на этапе после аутентификации.

Диагностика через Unified Logging выявила причину:

```
a1234@MacBook-Air-8 ~ % log stream --predicate 'process == "sshd"' --style compact
Filtering the log data using "process == "sshd""

2026-10-11 20:23:22.757 Df sshd[39209:1c043b] (libpam.2.dylib) in pam_sm_acct_mgmt(): pam_sacl: denying 'student' due to failed service ACL check
2026-10-11 20:23:22.764 Df sshd[39209:1c043b] fatal: Access denied for user student by PAM account configuration [preauth]
```

В macOS существует дополнительный уровень контроля доступа - Service Access Control List (SACL), реализуемый через PAM. По умолчанию доступ к SSH разрешен только членам группы admin. Пользователь student в нее не входил, поэтому sshd закрывал соединение даже при корректном ключе.

Решение:

```
a1234@MacBook-Air-8 ~ % sudo dseditgroup -o edit -a student -t user admin
a1234@MacBook-Air-8 ~ % id student
dscl . -read /Groups/admin GroupMembership | tr ' ' '\n' | grep -i student
uid=503(student) gid=20(staff) groups=20(staff),12(everyone),61(localaccounts),80(admin),701(com.apple.sharepoint.group.1),702(com.apple.sharepoint.group.2),33(_appstore),98(_lpadmin),100(_lpoperator),204(_developer),250(_analyticsusers),395(com.apple.access_ftp),398(com.apple.access_screensharing),399(com.apple.access_ssh),400(com.apple.access_remote_ae),101(access_bpf)
```

Попытка подключения к порту 2222 из нового конфига завершилась ошибкой:

```
a1234@MacBook-Air-8 ~ % ssh -p 2222 -o StrictHostKeyChecking=no -i ~/.ssh/id_ed25519_student \
    student@127.0.0.1 whoami
ssh: connect to host 127.0.0.1 port 2222: Connection refused
a1234@MacBook-Air-8 ~ % sudo lsof -iTCP -sTCP:LISTEN | grep -E 'ssh|launchd' | head
launchd       1  root   43u  IPv6 0xc619595bd086956b      0t0  TCP *:ssh (LISTEN)
launchd       1  root   44u  IPv4 0xc61959609d31f98b      0t0  TCP *:ssh (LISTEN)
launchd       1  root   45u  IPv6 0xc619595bd086956b      0t0  TCP *:ssh (LISTEN)
launchd       1  root   46u  IPv4 0xc61959609d31f98b      0t0  TCP *:ssh (LISTEN)
```

В macOS используется socket activation: launchd владеет сокетом на порту 22, а параметр Port 2222 в sshd_config не имеет эффекта, поскольку sshd не управляет сокетом. Директива Port в конфиге игнорируется - соединение к 2222 отклоняется на уровне ядра (порт никто не слушает). Смена порта в macOS требует правки plist-файла launchd-сервиса, что выходит за рамки штатной конфигурации.

Поскольку пункт «Port 2222» в задании помечен как опциональный, было принято решение убрать эту строку из конфига и продолжить работу с портом 22 по умолчанию. Все остальные параметры безопасности (запрет root, запрет пароля, ключевая аутентификация, MaxAuthTries, LoginGraceTime, AllowUsers) сохранены и работают.

```
a1234@MacBook-Air-8 ~ % sudo grep -n 'Port 2222' /etc/ssh/sshd_config
Password:
130:Port 2222
a1234@MacBook-Air-8 ~ % sudo sed -i '' '/^Port 2222$/d' /etc/ssh/sshd_config
a1234@MacBook-Air-8 ~ % sudo grep -n 'Port 2222' /etc/ssh/sshd_config
a1234@MacBook-Air-8 ~ % sudo sshd -T | grep -E '^(port|permitrootlogin|passwordauthentication|pubkeyauthentication|maxauthtries|logingracetime|allowusers)'
Error reading managed configuration (2: No such file or directory). Proceeding with default configuration.
port 22
logingracetime 30
maxauthtries 3
permitrootlogin no
pubkeyauthentication yes
passwordauthentication no
allowusers student
allowusers auditor
```

Перезапускаем SSH (на всякий случай, чтобы конфиг точно подхватился)

```
a1234@MacBook-Air-8 ~ % sudo launchctl unload /System/Library/LaunchDaemons/ssh.plist
a1234@MacBook-Air-8 ~ % sudo launchctl load -w /System/Library/LaunchDaemons/ssh.plist
a1234@MacBook-Air-8 ~ % sleep 2
a1234@MacBook-Air-8 ~ % sudo lsof -iTCP:22 -sTCP:LISTEN
COMMAND PID USER   FD   TYPE             DEVICE SIZE/OFF NODE NAME
launchd   1 root    8u  IPv6 0xc619595bd086956b      0t0  TCP *:ssh (LISTEN)
launchd   1 root   34u  IPv4 0xc61959609d9a176b      0t0  TCP *:ssh (LISTEN)
launchd   1 root   43u  IPv6 0xc619595bd086956b      0t0  TCP *:ssh (LISTEN)
launchd   1 root   44u  IPv4 0xc61959609d9a176b      0t0  TCP *:ssh (LISTEN)

a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -i ~/.ssh/id_ed25519_student \
    student@127.0.0.1 whoami
student
```

Ключ подхватился, вход по ключу работает без пароля.

4. Убедиться, что вход по паролю больше не работает

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o PreferredAuthentications=password \
    student@127.0.0.1
student@127.0.0.1: Permission denied (publickey,keyboard-interactive).
```

6.3 Проверка защиты SSH

1. Попытка входа под root

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o PreferredAuthentications=publickey \
    root@127.0.0.1
root@127.0.0.1: Permission denied (publickey,keyboard-interactive).
```

2. Попытка входа по паролю

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o PreferredAuthentications=password,keyboard-interactive \
    student@127.0.0.1
student@127.0.0.1: Permission denied (publickey).
```

3. Попытка входа с неверным ключом

Сначала сгенерируем «левый» ключ:

```
a1234@MacBook-Air-8 ~ % ssh-keygen -t ed25519 -f /tmp/wrong_key -N "" -C "wrong@test"
Generating public/private ed25519 key pair.
Your identification has been saved in /tmp/wrong_key
Your public key has been saved in /tmp/wrong_key.pub
The key fingerprint is:
SHA256:vIO91lfR+qHqqf7CL39hR37/OqSpaHUqqXmlKkGCNx4 wrong@test
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|                 |
| .             . |
|. E .  .      ...|
| o =    S     oo |
|  . .  o .o .o++.|
|     .. =* o.=+.+|
|    .  .BBo =o. o|
|     .+*++XXo .oo|
+----[SHA256]-----+
```

Попытка входа с ним:

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o PreferredAuthentications=publickey \
    -i /tmp/wrong_key student@127.0.0.1
student@127.0.0.1: Permission denied (publickey).
```

4. Проверка журнала SSH

В macOS нет journalctl. Аналог - log show:

```
a1234@MacBook-Air-8 ~ % log show --predicate 'process == "sshd"' --last 15m --style compact | tail -50

2026-10-11 20:41:36.249 Df sshd[39459:1c34c2] (libpam.2.dylib) in pam_sm_authenticate(): OpenDirectory - The authtok is incorrect.

2026-10-11 20:41:38.080 Df sshd[39457:1c34bc] error: PAM: authentication error for student from 127.0.0.1

2026-10-11 20:42:03.418 Df sshd[39457:1c34bc] fatal: Timeout before authentication for 127.0.0.1 port 52899
```

Эти записи подтверждают:

1) pam_sm_authenticate(): OpenDirectory - The authtok is incorrect - сервер отверг неверный пароль (работает PasswordAuthentication no);
2) fatal: Timeout before authentication - сработал параметр LoginGraceTime 30: sshd прервал сессию через 30 секунд, поскольку аутентификация не завершилась в отведенное время.

Успешная аутентификация по ключу:

```
a1234@MacBook-Air-8 ~ % sudo log show --predicate 'process == "sshd"' --last 5m --info --debug --style compact \
    | grep -iE 'Accepted|session opened|publickey'
Password:
2026-10-11 20:50:31.320 I  sshd[39551:1c4e81] Accepted publickey for student from 127.0.0.1 port 57355 ssh2: ED25519 SHA256:IQmTHX5uXPhyjPmQK0f/5O9z8KOsJFKw7+OVxCPcHFQ
```

5. Установка fail2ban и его аналогов

В Linux для защиты от брутфорса используется fail2ban. Однако это инструмент, специфичный для Linux, и его команды (apt, systemctl) в macOS неприменимы.

Установим sshguard. sshguard - это легковесный аналог fail2ban, который лучше интегрирован с системой безопасности macOS, в частности с брандмауэром pf.

```
a1234@MacBook-Air-8 ~ % brew install sshguard

a1234@MacBook-Air-8 ~ % sudo brew services start sshguard
Password:
Warning: Taking root:admin ownership of some sshguard paths:
  /opt/homebrew/Cellar/sshguard/2.5.1/sbin
  /opt/homebrew/Cellar/sshguard/2.5.1/sbin/sshguard
  /opt/homebrew/opt/sshguard
  /opt/homebrew/opt/sshguard/sbin
  /opt/homebrew/var/homebrew/linked/sshguard
This will require manual removal of these paths using `sudo rm` on
brew upgrade/reinstall/uninstall.
==> Successfully started `sshguard` (label: sh.brew.sshguard)
a1234@MacBook-Air-8 ~ % sudo brew services list | grep sshguard
sshguard started          root /Library/LaunchDaemons/sh.brew.sshguard.plist
```

Интеграция с pf. В конец /etc/pf.conf добавлены правила:

```
a1234@MacBook-Air-8 ~ % sudo tee -a /etc/pf.conf > /dev/null <<'EOF'
block in quick on { en0, lo0 } proto tcp from <sshguard> to any port 22 label "ssh bruteforce"
EOF
a1234@MacBook-Air-8 ~ % sudo tail -4 /etc/pf.conf
pass in proto udp from any to any port 5354

table <sshguard> persist
block in quick on { en0, lo0 } proto tcp from <sshguard> to any port 22 label "ssh bruteforce"
```

Первое правило объявляет таблицу <sshguard> - динамический список IP-адресов, подлежащих блокировке.

Второе - блокирует любой входящий TCP-трафик на порт 22 от адресов из этой таблицы. Указаны два интерфейса: en0 (Wi-Fi) и lo0 (loopback).

Применение:

```
Status: Disabled for 0 days 04:21:01
a1234@MacBook-Air-8 ~ % sudo pfctl -e
No ALTQ support in kernel
ALTQ related functions disabled
pf enabled

a1234@MacBook-Air-8 ~ % sudo pfctl -f /etc/pf.conf
pfctl: Use of -f option, could result in flushing of rules
present in the main ruleset added by the system at startup.
See /etc/pf.conf for further details.

No ALTQ support in kernel
ALTQ related functions disabled
```

```
a1234@MacBook-Air-8 ~ % sudo pfctl -s rules | grep -i sshguard
No ALTQ support in kernel
ALTQ related functions disabled
block drop in quick on en0 proto tcp from <sshguard> to any port = 22 label "ssh bruteforce"
```

Первоначальная версия правила содержала директиву on en0, что ограничивало область его действия физическим интерфейсом Wi-Fi. Поскольку проверка выполнялась на localhost (127.0.0.1), трафик шел через loopback-интерфейс lo0 и под действие правила не попадал. После добавления lo0 в список интерфейсов правило стало применяться и к loopback-трафику. Это иллюстрирует важную особенность pf: правила по умолчанию привязываются к конкретному интерфейсу.

Верификация механизма блокировки. Сначала проверено, что SSH работает штатно:

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
    -i ~/.ssh/id_ed25519_student student@127.0.0.1 whoami
student
```

Затем IP-адрес 127.0.0.1 добавлен в таблицу <sshguard>:

```
a1234@MacBook-Air-8 ~ % sudo pfctl -t sshguard -T add 127.0.0.1
No ALTQ support in kernel
ALTQ related functions disabled
1/1 addresses added.
a1234@MacBook-Air-8 ~ % sudo pfctl -t sshguard -T show
No ALTQ support in kernel
ALTQ related functions disabled
   127.0.0.1
```

После добавления IP в блок-лист SSH-подключение с него даже с валидным ключом стало невозможно:

```
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -o ConnectTimeout=5 \
    -i ~/.ssh/id_ed25519_student student@127.0.0.1 whoami
ssh: connect to host 127.0.0.1 port 22: Operation timed out
```

Связка pf + <sshguard> блокирует пакеты на уровне ядра - до их доставки процессу sshd. При автоматической работе sshguard эта таблица наполняется динамически, обеспечивая защиту от брутфорса без ручного вмешательства.

Разблокировка выполняется командой pfctl -t sshguard -T flush, после чего доступ восстанавливается:

```
a1234@MacBook-Air-8 ~ % sudo pfctl -t sshguard -T flush
No ALTQ support in kernel
ALTQ related functions disabled
1 addresses deleted.
a1234@MacBook-Air-8 ~ % sudo pfctl -t sshguard -T show
No ALTQ support in kernel
ALTQ related functions disabled
a1234@MacBook-Air-8 ~ % ssh -o StrictHostKeyChecking=no -i ~/.ssh/id_ed25519_student student@127.0.0.1 whoami
student
```

Проверка автоматического срабатывания sshguard. Было выполнено 5 неудачных попыток SSH-аутентификации подряд с посторонним ключом:

```
a1234@MacBook-Air-8 ~ % for i in 1 2 3 4 5; do
  ssh -o StrictHostKeyChecking=no -o PreferredAuthentications=publickey \
      -i /tmp/wrong_key student@127.0.0.1 whoami 2>&1 | tail -1
  echo "--- attempt $i ---"
done
student@127.0.0.1: Permission denied (publickey).
--- attempt 1 ---
student@127.0.0.1: Permission denied (publickey).
--- attempt 2 ---
student@127.0.0.1: Permission denied (publickey).
--- attempt 3 ---
student@127.0.0.1: Permission denied (publickey).
--- attempt 4 ---
student@127.0.0.1: Permission denied (publickey).
--- attempt 5 ---
a1234@MacBook-Air-8 ~ % sudo pfctl -t sshguard -T show
No ALTQ support in kernel
ALTQ related functions disabled
```

Таблица <sshguard> при этом осталась пустой. Причина: sshguard не умеет разбирать Unified Logging (новый формат журнала macOS), а традиционные текстовые логи (/var/log/auth.log, /var/log/secure), к которым он приспособлен, в современных версиях macOS отсутствуют. Это документированное ограничение sshguard на macOS; для полноценной автоматизации требуется сторонний парсер Unified Logging, преобразующий записи sshd в формат, понятный sshguard.

Контрольные вопросы

1. **Как работает мандатный контроль доступа в macOS? Чем отличается от SELinux / AppArmor?**

В macOS мандатный контроль доступа реализован через механизм Sandbox (Seatbelt) и утилиту sandbox-exec. Профили пишутся на языке SBPL (Sandbox Profile Language). Профиль явно разрешает или запрещает доступ к файлам, сети, системным вызовам и т.д. При

запуске процесса с профилем ядро принудительно применяет ограничения.

Отличия:

1. В отличие от AppArmor (Ubuntu), в macOS нет режима complain - профиль сразу работает в режиме enforce. Директива (trace) игнорируется.
2. В отличие от SELinux, sandbox-профили обычно привязаны к конкретному приложению/процессу, а не работают как глобальная система меток (MLS/MCS). SELinux сложнее и мощнее, sandbox проще и ориентирован на изоляцию отдельных приложений.
3. Apple постепенно переходит на EndpointSecurity framework, а sandbox-exec помечен как deprecated, хотя пока полностью работает.

2. Для чего используется аудит в macOS? Приведите пример.

Аудит нужен для регистрации важных событий безопасности: создание/удаление пользователей, выполнение sudo, изменение критических файлов, аутентификация и т.д.

В macOS классический OpenBSM (auditd) устарел (с macOS 11, отключён с macOS 14). Рекомендуемый механизм -Unified Logging.

Пример (аналог auditctl -w /etc/passwd -p wa):

```
# Мониторинг в реальном времени событий управления пользователями

log stream --predicate 'process == "sysadminctl" OR process == "dscl" OR eventMessage CONTAINS "user"' --info

# Историческая выборка

log show --predicate 'eventMessage CONTAINS "Creating user record" OR eventMessage CONTAINS "Deleting record"' --last 1h
```

Аналог правила на sudo:

```
log stream --predicate 'process == "sudo"' –info
```

3. Как настроить автоматические обновления безопасности в macOS?

Через службу Software Update (softwareupdated) и plist:

```
# Включить автоматическую установку обновлений безопасности

sudo defaults write
/Library/Preferences/com.apple.SoftwareUpdate
AutomaticallyInstallMacOSUpdates -bool true

sudo defaults write
/Library/Preferences/com.apple.SoftwareUpdate
CriticalUpdateInstall -bool true

sudo defaults write
/Library/Preferences/com.apple.SoftwareUpdate
ConfigDataInstall -bool true

# Проверить текущие настройки

defaults read /Library/Preferences/com.apple.SoftwareUpdate
```

Проверка доступных обновлений:

```
softwareupdate –list
```

4. Почему рекомендуется отключать вход root по SSH?

Root имеет неограниченные привилегии. При компрометации учетной записи злоумышленник сразу получает полный контроль над системой.

У root часто предсказуемое имя (всегда root), что упрощает брутфорс.

Лучшая практика - вход под обычным пользователем + повышение прав через sudo (с возможностью логирования и ограничений).

В macOS дополнительно работает SACL (Service Access Control List) - по умолчанию SSH разрешен только членам группы admin / com.apple.access_ssh.

5. В чем преимущество ключевой аутентификации перед парольной?

Парольная	Ключевая
Можно подобрать (брутфорс, словарные атаки)	Практически невозможно подобрать при достаточной длине ключа
Пароль передается (даже в хеше)	Приватный ключ никогда не покидает клиент
Легко украсть/перехватить	Можно дополнительно защитить passphrase + агентом
Слабая против фишинга	Устойчива к фишингу (нет «ввода пароля»)

6. Что такое аналог fail2ban в macOS и как он защищает SSH?

fail2ban в чистом виде плохо работает на современных macOS (нет классических /var/log/auth.log). Аналог - sshguard + pf (Packet Filter):

sshguard анализирует логи неудачных попыток входа. При превышении порога добавляет IP в таблицу pf <sshguard>. pf блокирует весь трафик с этих IP на порт 22 на уровне ядра (до того, как пакет дойдет до sshd).

Пример правил в /etc/pf.conf:

```
table <sshguard> persist

block in quick on { en0 lo0 } proto tcp from <sshguard> to any port 22
```

7. Как проверить, что брандмауэр (pf / Application Firewall) блокирует входящие соединения на запрещенные порты?

1) Создать правило блокировки порта (например, 80):

```
echo "block in proto tcp to any port 80" | sudo pfctl -f -

sudo pfctl -e
```

2) В одном терминале запустить «сервер»:

```
nc -l 80
```

3) В другом терминале попытаться подключиться:

```
nc -v 127.0.0.1 80
```

Соединение должно быть отклонено (Connection refused / timeout).

4) Проверить загруженные правила:

```
sudo pfctl -s rules
```

Также можно использовать Application Firewall + Stealth Mode (компьютер не отвечает на ping и нежелательные входящие запросы).

8. Какие параметры sshd_config отвечают за ограничение числа попыток входа?

Основные параметры:

Параметр	Назначение	Рекомендуемое значение
MaxAuthTries	Максимальное число попыток аутентификации в одной сессии	3
LoginGraceTime	Время (в секундах), за которое нужно успеть пройти аутентификацию	30

MaxStartups	Ограничение на количество одновременных незавершенных соединений	10:30:60
AllowUsers / AllowGroups	Разрешить вход только определенным пользователям/группам	student auditor
PasswordAuthentication	Запрет парольной аутентификации	no
PermitRootLogin	Запрет входа под root	no

После изменения конфига в macOS нужно перезапустить службу:

```
sudo launchctl kickstart -k system/com.openssh.sshd
```

Таблица 1. Настройки безопасности

Средство / Задание	В Linux (по заданию)	Аналог / Реализация в macOS	Результат настройки
1. Пользователи и права	adduser, groupadd, usermod, chown, chmod, passwd, umask, chattr/lsattr	dscl / sysadminctl; dscl для групп; chown/chmod; passwd; umask; chflags / ls -lO	Созданы student, auditor; группа security; файл 640; umask 027; флаг uchg работает
2. Брандмауэр (UFW)	ufw default deny incoming; ufw allow 22/tcp; ufw deny 80/tcp	Application Firewall + Stealth Mode; pf (Packet Filter) с правилами block in proto tcp to any port 80	Firewall + Stealth включены; порт 80 заблокирован (соединение отклоняется)
3. Аудит (auditd)	auditd + auditctl -w /etc/passwd -p wa; auditctl -w /usr/bin/sudo -p x; ausearch	OpenBSM (устарел) + Unified Logging (log stream / log show); фильтры по process и eventMessage	OpenBSM не пишет новые trail без reboot; события создания пользователя и sudo зафиксированы через log show/stream
4. Мандатный контроль (AppArmor)	aa-status; aa-complain / aa-enforce; профили в /etc/apparmor.d/	sandbox-exec (Seatbelt) + SBPL-профили; режим enforce работает, complain отсутствует	Профиль блокирует чтение /tmp/secret.txt и запись /tmp/testapp_output; логи в Unified Logging (Sandbox)
5. Автообновления безопасности	unattended-upgrades; /etc/apt/apt.conf.d/50unattended-upgrades	softwareupdated; defaults read /Library/Preferences/com.apple.SoftwareUpdate	Автопроверка работает (обнаружен апдейт Sonoma 14.8.9); AutomaticallyInstallMacOSUpdates включен
6. Защита SSH	sshd_config: PermitRootLogin no, PasswordAuthentication no, Port 2222, MaxAuthTries 3, LoginGraceTime 30, AllowUsers; fail2ban	Аналогичные параметры в /etc/ssh/sshd_config; socket activation (launchd); SACL (группа com.apple.access_ssh); sshguard + pf	Ключевая аутентификация работает; пароль и root запрещены; MaxAuthTries/LoginGraceTime активны; Port 2222 не работает (socket activation); sshguard не парсит Unified Logging

Таблица 2. Найденные события и результаты проверок

Источник	Событие / Действие	Что зафиксировано	Вывод / Значение
Unified Logging (log stream / show)	Создание / удаление пользователя testuser	Removing testuser's home; Deleting record; Creating user record; Assigning UID: 505 GID: 20; Home directory assigned	Полный жизненный цикл учётной записи зафиксирован синхронно с командами
Unified Logging	Выполнение sudo (whoami, id, ls, date)	Отдельный процесс sudo для каждого вызова; записи с timestamp	Эквивалент auditctl -w /usr/bin/sudo -p x; все вызовы отслеживаются
Sandbox (Unified Logging)	Запуск testapp.sh под enforce-профилем	DENIED на чтение /tmp/secret.txt и создание /tmp/testapp_output	Мандатный контроль работает; логи ядра доступны через log stream
sshd + Unified Logging	Неудачная аутентификация по паролю	pam_sm_authenticate(): The authtok is incorrect	PasswordAuthentication no работает корректно
sshd + Unified Logging	Таймаут аутентификации	fatal: Timeout before authentication	LoginGraceTime 30 срабатывает
sshd	Успешный вход по ключу student	Accepted publickey for student	Ключевая аутентификация и AllowUsers работают
pf + sshguard	Ручное добавление 127.0.0.1 в таблицу <sshguard>	SSH-соединение с валидным ключом блокируется на уровне ядра	Механизм блокировки pf работает; автоматика sshguard ограничена (не парсит Unified Logging)
Application Firewall / pf	Попытка подключения к порту 80	Соединение отклоняется (Connection refused / timeout)	Правило block in proto tcp to any port 80 эффективно
chflags / ls -lO	Установка флага uchg на файл	Файл становится неизменяемым (даже для root без снятия флага)	Аналог chattr +i работает через chflags